Νευρωνικά Δίκτυα – Βαθιά Μάθηση : Εργασία 3.

Δημήτριος Μουταφτσίδης , 9920 (ΤΗΜΜΥ)

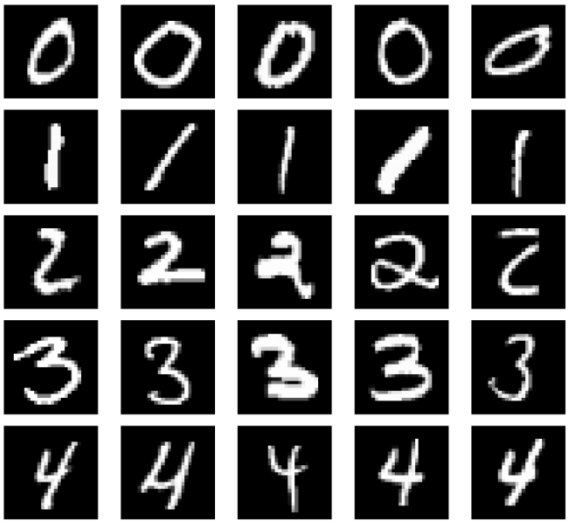
moutdimi@ece.auth.gr

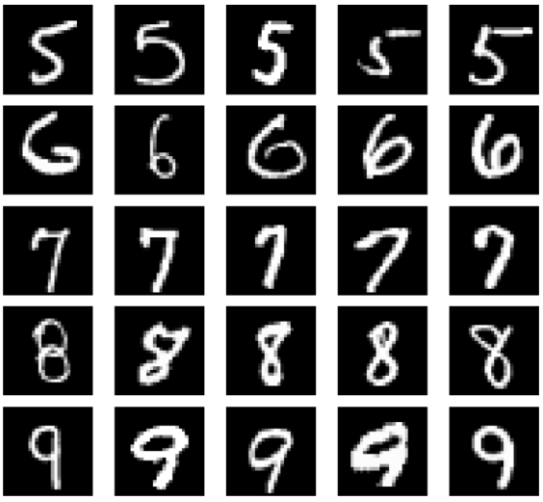
1. Εισαγωγή.

Η παρούσα εργασία υλοποιεί εκπαίδευση Autoencoder για ανακατασκευή ψηφίων της βάσης δεδομένων MNIST. Υλοποιούνται δύο αρχιτεκτονικές. MLP και Convolutional Autoencoder. Γίνεται σύγκριση των αποτελεσμάτων με ανακατασκευή μέσω PCA. Περιλαμβάνει επίσης ένα CNN, εκπαιδευμένο στην ταξινόμηση της MNIST. Χρησιμοποιείται πειραματικά για αξιολόγηση των reconstructions. Πέραν της απλής ανακατασκευής, υλοποιείται και ανακατασκευή επόμενου ψηφίου. Ο σχολιασμένος κώδικας βρίσκεται στο αρχείο Autoencoder.ipynb. Εκεί σχολιάζονται οι τεχνικές λεπτομέρειες της υλοποίησης. Γίνεται χρήση Python. Συγκεκριμένα PyTorch κατά κύριο λόγο. Στην παρούσα αναφορά γίνεται σύντομη περιγραφή της θεωρίας πίσω από τα μοντέλα. Παρουσιάζονται επίσης τα πειράματα εκπαίδευσης και αξιολόγησης.

2. Η Βάση Δεδομένων MNSIT.

Κλασσικό εκπαιδευτικό dataset. Επιλέχθηκε για την υλοποίηση των Autoencoders (εφ' εξής AEs) καθώς κρίναμε πως είναι πιο απλή και ελεγχόμενη για πειράματα ανακατασκευής. Περιλαμβάνει 60.000 grayscale εικόνες διάστασης 28x28 των ψηφίων 0 έως 9. Ας δούμε ορισμένες εξ αυτών:





Το μονότονο background και η έντονη χρωματική αντίθεση είναι θετικά πρώτα δείγματα. Το dataset περιλαμβάνει designated test set. Για να αποφύγουμε , ωστόσο, το information leakage που συζητήσαμε στην προηγούμενη εργασία, αποκόπτουμε ένα μέρος του training set και το ορίζουμε ως validation. Επιλέχθηκε μέγεθος 10.000. Περίπου 16% του αρχικού. Θεωρήσαμε πως αφήνει χώρο για αποτελεσματική εκπαίδευση, ενώ επιτρέπει παράλληλα καλή αξιολόγηση. Δεδομένης της ύπαρξης test set, θεωρήσαμε πως η οδηγία περί 60% - 40% split δεν εμπίπτει στην προκειμένη περίπτωση. Ακόμη, ο διαχωρισμός γίνεται με stratified τρόπο, ώστε να διατηρηθεί η αναλογία των κλάσεων. Ας δούμε λοιπόν την συχνότητα εμφάνισης της κάθε κλάσης:

```
Training class counts: {0: 4936, 1: 5618, 2: 4965, 3: 5109, 4: 4868, 5: 4518, 6: 4932, 7: 5221, 8: 4876, 9: 4957}
Training class percentages: {0: 9.87, 1: 11.24, 2: 9.93, 3: 10.22, 4: 9.74, 5: 9.04, 6: 9.86, 7: 10.44, 8: 9.75, 9: 9.91}
```

```
Validation class counts: {0: 987, 1: 1124, 2: 993, 3: 1022, 4: 974, 5: 903, 6: 986, 7: 1044, 8: 975, 9: 992}
Validation class ratios: {0: 9.87, 1: 11.24, 2: 9.93, 3: 10.22, 4: 9.74, 5: 9.03, 6: 9.86, 7: 10.44, 8: 9.75, 9: 9.92}
```

Οι αναλογίες διατηρήθηκαν επιτυχώς. Το dataset έχει έτσι κι αλλιώς σχεδόν ομοιόμορφη κατανομή.

3. Autoencoders.

3.1. Γενικά για AEs.

Πρόκειται για <u>ειδική περίπτωση NN</u>. Αποσκοπεί στην ανακατασκευή του input, όσο το δυνατόν πιο πιστά. Χωρίς, όμως, στεγνή απομνημόνευση των δεδομένων. Ιδανικά, μαθαίνει έναν σχετικά μικρό αριθμό features σε σχέση με τον όγκο του αρχικού input. Για παράδειγμα, στην δική μας περίπτωση οι εικόνες αποτελούνται από 28x28 = 784 pixels. <u>Ψάχνουμε ένα latent dimension τέτοιο ώστε να ισορροπεί meaningful compression και ικανότητα ανακατασκευής</u>. Η

ισορροπία αυτή εξαρτάται από το εκάστοτε task. Μέσω του mapping που μαθαίνει, θέλουμε να είναι σε θέση να αναπαράγει instances της κάθε κλάσης. Ένας «καλός» Autoencoder μαθαίνει τα πιο εκφραστικά features. Αυτά δηλαδή που περιέχουν την περισσότερη πληροφορία.

Βασική Ιδέα:

Encoder f_e: $z = f_e(x)$

Decoder g_d: $\hat{x} = g_d(z)$

Δηλαδή: $\hat{x} = g_d(f_e(x))$

Η εκπαίδευση γίνεται με βάση την ελαχιστοποίηση του reconstruction loss. Στην περίπτωσή μας του MSE μεταξύ αρχικών και reconstructed pixels. Αποσκοπεί στην προσέγγιση των δύο παραπάνω συναρτήσεων συμπίεσης – αποσυμπίεσης.

Κατά το training run έχουμε:

Forward pass: $x \rightarrow z \rightarrow \hat{x}$

Loss: $MSE(x, \hat{x})$

Back Propagation: $Dec \rightarrow Enc$

Weight Updates: μέσω Adam στην παρούσα υλοποίηση.

Παρατηρούμε πως δεν εμπλέκονται κάπου labels. Τα targets ταυτίζονται με το input. Πρόκειται για περίπτωση self-supervised learning**.**

Η διαδικασία φαντάζει αρχικά κάπως trivial. Γιατί να εκπαιδεύσει κανείς AE; Που μπορεί να χρησιμεύσει η αναπαραγωγή του input;

1. Σε representation learning: Συμπιεσμένη αναπαράσταση του μεγαλύτερου μέρους της πληροφορίας των αρχικών δεδομένων.

2. Denoising: Ειδική κατηγορία των AEs που ανακατασκευάζει αποθορυβοποιημένες εκδοχές των δεδομένων. Ο θόρυβος δεν βρίσκεται στα πιο informative features.

3. Feature Extraction: Η latent αναπαράσταση z μπορεί να χρησιμεύσει σε classification tasks. Παρέχει πιο συμπυκνωμένη πληροφορία.

3.2.1. MLP Autoencoders.

Η πιο απλή τους μορφή. Χρησιμοποιεί Fully Connected Dense Layers για την κωδικοποίηση και την αποκωδικοποίηση. Έχει περιορισμένο learning capacity στην περίπτωση των 2-D εικόνων. Δέχεται άλλωστε flattened input. Αυτό, ωστόσο, δεν σημαίνει πως δεν είναι σε θέση να μάθει καθόλου patterns. Ακολουθεί παράδειγμα για τέσσερα dense layers:

$$z = \sigma(W_2\sigma(W_1x + b_1) + b_2)$$

$$\hat{x} = \sigma(W_4\sigma(W_3z + b_3) + b_4)$$

3.2.2. Ο δικός μας MLP Autoencoder (Class Melchior).

Η αρχιτεκτονική που επιλέξαμε αφήνει το μέγεθος του latent dimension ως την μόνη προς πειραματισμό παράμετρο. Πρόκειται για το τελικό μέγεθος (αριθμός features) στην έξοδο του encoder και για το αρχικό μέγεθος στην είσοδο του decoder. Η υπόλοιπη σταθερή δομή καθορίστηκε από γρήγορες δοκιμές που δεν περιλαμβάνονται στον κώδικα. Ο MLP, έτσι κι αλλιώς, περιλαμβάνεται για λόγους πληρότητας κυρίως.

Αρχιτεκτονική Encoder:

1. $Linear\ Procection(784 \rightarrow 256)$

2. $1D\ Batch\ Normalization(256)$

3. $ReLU$

4. $Linear\ Projection(256 \rightarrow 128)$

5. $1D\ Batch\ Normalization(128)$

6. $ReLU$

7. $Linear\ Projection(128 \rightarrow latent\ dimension)$

Τα ορίσματα των FCL αποτελούν design parameter και επηρεάζουν την επίδοση του μοντέλου. Το ίδιο και η παρουσία Batch Norm. Η ReLU activation εισάγει non-linearity. Είναι απαραίτητη για την εκμάθηση «ζωντανού» mapping.

Αρχιτεκτονική Decoder:

1. $Linear\ Procection(latent\ dimension \rightarrow 128)$

2. $1D\ Batch\ Normalization(128)$

3. $ReLU$

4. $Linear\ Projection(128 \rightarrow 256)$

5. $1D\ Batch\ Normalization(256)$

6. $ReLU$

7. $Linear\ Projection(256 \rightarrow 784)$

8. $Sigmoid$

Η παρουσία της Sigmoid εξασφαλίζει πως τα output values βρίσκονται στο εύρος [0,1]. Όπως ακριβώς το αρχικό input για την κανονικοποιημένη MNIST.

3.3.1 Convolutional Autoencoders.

Ό,τι έχουμε αναφέρει για τις βασικές αρχές των AEs ισχύει. Κρίσιμη διαφορά αποτελεί η χρήση Convolutional αντί για Fully Connected Layers. Όπως είχαμε συζητήσει και για τα CNN, η πράξη της συνέλιξης από την φύση της λαμβάνει υπόψη την χωρική δομή της εικόνας. Το μοντέλο λαμβάνει ως input την δισδιάστατη εικόνα. Επίσης μαθαίνει πολύ μικρότερο αριθμό παραμέτρων σε σχέση με τους MLP. Είναι ιδανικοί για image compression και reconstruction tasks.

3.3.2. Ο δικός μας Convolutional Autoencoder (μοντέλο Balthazar).

Οι προς τροποποίηση και πειραματισμό παράμετροι είναι το μέγεθος του latent dimension και το πλάτος του δικτύου στα ενδιάμεσα layers. Το μέγεθος του latent dimension επιλέγεται έμμεσα από το όρισμα channels και υπολογίζεται ως $7 \cdot 7 \cdot channels$. Το $7 \cdot 7$ προκύπτει από το τελικό μέγεθος των feature maps μετά από downsampling. Ανήκει στην σταθερή αρχιτεκτονική του μοντέλου, η οποία και πάλι καθορίστηκε από “off the record” πειράματα.

Αρχιτεκτονική Encoder:

1. $Convolution(1\,, 28 \times 28 \;\rightarrow\; initial\ width\,, 14 \times 14)$

2. $2D\ Batch\ Normalization(initial\ width)$

3. $ReLU$

4. $Convolution(initial\ width\,, 14 \times 14 \;\rightarrow 2 \cdot initial\ width\,, 7 \times 7)$

5. $2D\ Batch\ Normalization(2 \cdot initial\ width)$

6. $ReLU$

7. $Convolution(2 \cdot initial\ width\,, 7 \times 7 \;\rightarrow channels\,, 7 \times 7)$

8. $2D\ Batch\ Normalization(channels)$

9. $ReLU$

Αρχιτεκτονική Decoder:

1. $Convolution^T(channels\,, 7 \times 7 \;\rightarrow\; 2 \cdot initial\ width\,, 7 \times 7)$

2. $2D\ Batch\ Normalization(2 \cdot initial\ width)$

3. $ReLU$

4. $Convolution^T(2 \cdot initial\ width\,, 7 \times 7 \;\rightarrow initial\ width\,, 14 \times 14)$

5. $2D\ Batch\ Normalization(initial\ width)$

6. $ReLU$

7. $Convolution^T(initial\ width\,, 14 \times 14 \;\rightarrow inital\ width\ /\ 2\,, 28 \times 28)$

8. $2D\ Batch\ Normalization(initial\ width\ /\ 2)$

9. $ReLU$

10. $Convolution(initial\ width\ /\ 2\,, 28 \times 28 \;\rightarrow 1\,, 28 \times 28)$

11. $Sigmoid$

Το downsampling υλοποιείται μέσω strided convolutions αντί για max pooling. Η επιλογή αυτή καθιστά τη διαδικασία υποδειγματοληψίας learnable, επιτρέποντας στο δίκτυο να μάθει ποια χαρακτηριστικά πρέπει να διατηρηθούν ώστε να βελτιστοποιηθεί το reconstruction. Επιπλέον, η συμμετρική αντιστοίχιση με transpose convolutions στο decoder διευκολύνει τον σχεδιασμό του encoder–decoder pipeline.

4. Μοντέλα Σύγκρισης Επίδοσης.

Τα δύο ακόλουθα μοντέλα χρησιμοποιούνται βοηθητικά στην αξιολόγηση της επίδοσης των Autoencoders.

4.1. PCA Reconstruction (class Eva).

Θυμίζουμε για το PCA: πρόκειται για γραμμική μέθοδο που προσδιορίζει ορθοκανονικό σύνολο διανυσμάτων που αποτυπώνει τις κατευθύνσεις μεγαλύτερου variance (δηλαδή πληροφορίας). Είναι πολύ κάλο baseline για αξιολόγηση απόδοσης ενός AE. Εάν η πιο πολύπλοκη διαδικασία εκπαίδευσης και tuning του δεν αποδίδει καλύτερα από την απλή, γραμμική και γρήγορη ανακατασκευή μέσω PCA, τότε το μοντέλο καθίσταται περιττό. Επίσης, θα μπορούσαμε να πούμε ότι MLP Autoencoder με μόνο γραμμικούς μετασχηματισμούς και MSE loss criterion ανάγεται σε PCA reconstruction. Το PCA, μάλιστα, είναι η καλύτερη δυνατή γραμμική κωδικοποίηση δεδομένων. Έτσι αποτελεί καλό σύνορο μεταξύ γραμμικής και μη γραμμικής ανακατασκευής. Η encoding-decoding διαδικασία μέσω PCA συνοψίζεται ως εξής: συμπίεση (προβολή) στον χώρο των X

principal components, και στην συνέχεια προβολή πίσω στον αρχικό.

4.2. CNN για Αναγνώριση Ψηφίων (μοντέλο Kasper).

Ενδιαφέρον πείραμα αποτελεί η σύγκριση επίδοσης ταξινομητή της MNIST στα αρχικά και στα reconstructed ψηφία. Μπορούμε να χρησιμοποιήσουμε την διαφορά στην ακρίβεια ως μέτρο αξιολόγησης ποιότητας ανακατασκευής. Η αρχιτεκτονική που επιλέχθηκε περιλαμβάνει:

1. Δύο συνελικτικά blocks.

2. Batch Normalization

3. ReLU activation

4. Max Pooling

Δεν μπαίνουμε σε παραπάνω λεπτομέρειες καθώς έχουμε ήδη εξετάσει σε προηγούμενο παραδοτέο την δομή ενός CNN. Η επίδοση που πετυχαίνει το μοντέλο Kasper είναι εξαιρετική. Χρειάστηκε ελάχιστο tuning. Σε αυτό ευθύνεται, φυσικά, η καταλληλόλητα της MNIST.

5. Πειράματα Εκπαίδευσης Μοντέλων.

Στην παρούσα ενότητα θα δούμε τα πειράματα για τον tuning των μοντέλων. Εξετάζεται μόνο η μη σταθερή αρχιτεκτονική. Κάθε δοκιμή επιστρέφει:

1. Το μοντέλο.
2. Τα βάρη που σημείωσαν την καλύτερη επίδοση.
3. Dictionary με αποτελέσματα. Μεταξύ άλλων περιέχει τα logs κάθε epoch (per epoch training/validation loss/accuracy) , αρχικό LR, χρόνο εκπαίδευσης.

Τόσο για το CNN όσο και για τους AE, εκπαιδεύουμε αυστηρά πάνω στο training set και αξιολογούμε στο validation

5.1. Εκπαίδευση CNN.

Για το CNN παρακολουθούμε δύο metrics. Accuracy και loss.

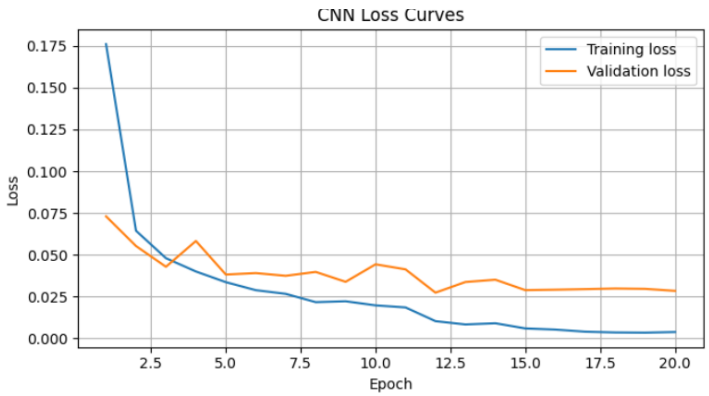
Setup:

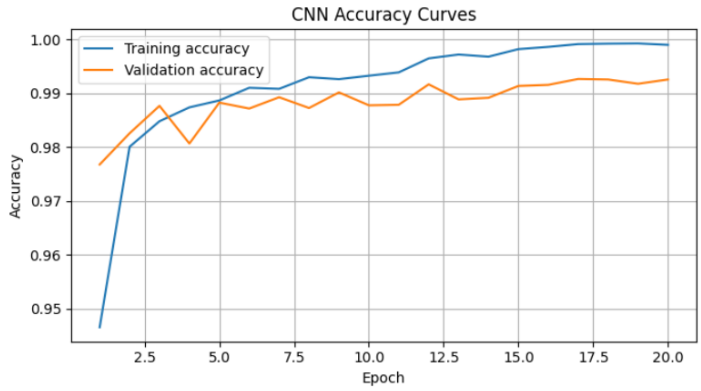
1. Optimizer: Adam με αρχικό LR 10^-3 και Weight Decay 10^-4.
2. Scheduler: Reduce On Plateau με patience = 2 και factor = 0.5.
3. Criterion: Cross Entropy Loss.
4. Epochs: 20

Αποτελέσματα:

1. Training Time: 10 mins
2. Best Validation Accuracy: 99,27%

Training/Validation Curves:





Συμπεράσματα:

Ελαφρύ, γρήγορο στην εκπαίδευση και πολύ ακριβές δίκτυο. Υγιή accuracy/loss curves. Δεν κρίθηκε απαραίτητο τα γίνουν παραπάνω πειράματα. Περισσότερος κόπος για trivial gains.

5.2. Εκπαίδευση AEs.

Η γενική φιλοσοφία του training run είναι η ίδια. Για τους AEs, όμως, εργαζόμαστε με μοναδικό κριτήριο το loss. Η συνάρτηση που υλοποιεί το training run υποστηρίζει εκπαίδευση και για απλό reconstruction και για next digit generation. Στην παρούσα φάση εκπαιδεύουμε για το πρώτο.

Καλό θα ήταν να αναφέρουμε πως οι επιλογές μεγέθους του Latent Dimension που θα δούμε στα πειράματα έγιναν κάπως αυθαίρετα. Προσπαθήσαμε να παράξουμε μοντέλα που συνδυάζουν αξιοπρεπή επίδοση και meaningful συμπίεση. Κύριος σκοπός των πειραμάτων είναι το να εξετάσουμε το πως οι μεταβολές επηρεάζουν το μοντέλο.

5.2.1. Εκπαίδευση MLP Autoencoder.

Έγιναν τρία πειράματα. Η μόνη παράμετρος που τροποποιήθηκε είναι το μέγεθος του latent dimension.

Σταθερά για τα τρία πειράματα:

1. Optimizer: Adam με αρχικό LR 10^-3.
2. Loss Criterion: MSE
3. Scheduler: Reduce On Plateau με patience = 0 , factor = 0.5 , ελάχιστο LR = 10^-6 , cooldown = 1 , threshold = 10^-4.
4. Epochs: 30

Δεν προσθέσαμε Weight Decay στον Adam. Δοκιμάστηκε σε off-the-record μικρής κλίμακας πειράματα. Δεν φάνηκε να βοηθάει. Ο Scheduler επιλέχθηκε να είναι λίγο πιο επιθετικός, ώστε να βοηθήσει στην σύγκλιση.

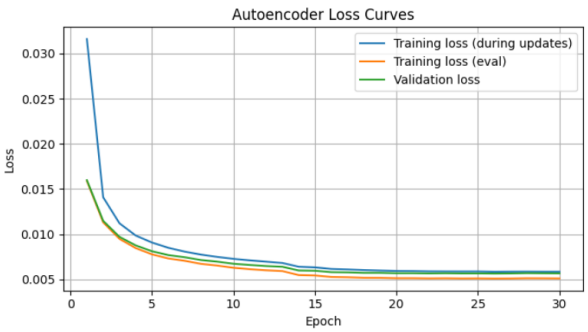
Setup 1:

Latent dimension μεγέθους 32.

Αποτελέσματα:

1. Minimum Loss: 0.005654
2. Training time: 18 λεπτά

Training/Training in .eval() mode/Validation Curves:



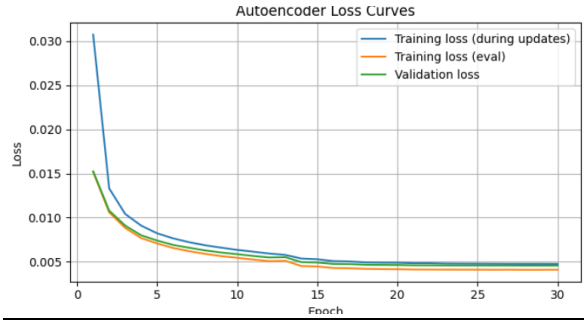
Setup 2:

Latent dimension μεγέθους 64.

Αποτελέσματα:

1. Minimum Loss: 0.004563
2. Training Time: 17,5 λεπτά

Training/Training in .eval() mode/Validation Curves:



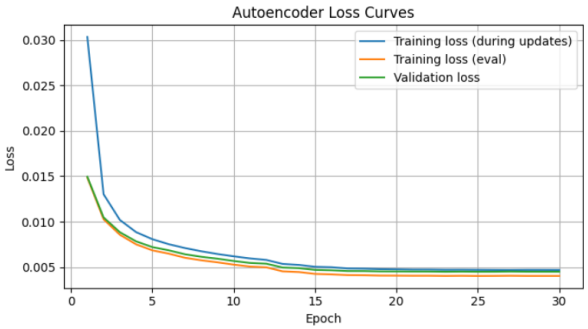
Setup 3:

Latent dimension μεγέθους 128.

Αποτελέσματα:

1. Minimum Loss: 0.004483
2. Training Time: 17,5 λεπτά

Training/Training in .eval() mode/Validation Curves:



Συμπεράσματα:

Η αύξηση του latent dimension φαίνεται να μειώνει marginally το loss. Δεν φαίνεται να επηρεάζουν το training time. Τα μοντέλα έχουν συγκλίνει επαρκώς.

5.2.2. Εκπαίδευση Convolutional Autoencoders.

Επίσης τρία πειράματα. Κάναμε pivot μεταξύ αύξησης latent dimension (μέσω των channels) και αύξησης του πλάτους του δικτύου όπως θα φανεί παρακάτω.

Σταθερά για τα τρία πειράματα:

1. Optimizer: Adam με αρχικό LR 10^-3.
2. Loss Criterion: MSE
3. Scheduler: Reduce On Plateau με patience = 0 , factor = 0.5 , ελάχιστο LR = 10^-6 , cooldown = 1 , threshold = 10^-4.
4. Epochs: 30

Setup 1:

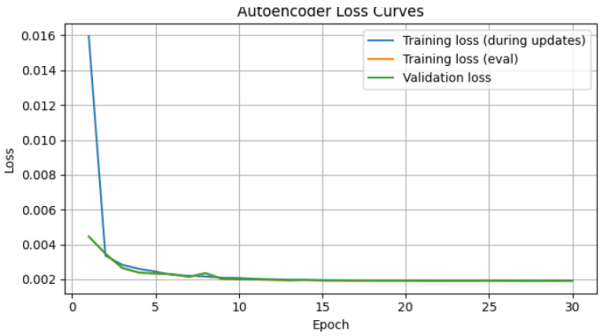
1. Latent dimension μεγέθους 98
2. Width 32

Αποτελέσματα:

1. Minimum Loss: 0.001927

2. Training Time: 17 λεπτά

Training/Training in .eval() mode/Validation Curves:



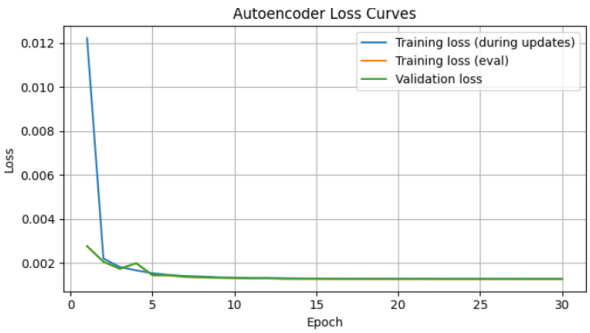
Setup 2:

1. Latent dimension μεγέθους 147
2. Width 32

Αποτελέσματα:

1. Minimum Loss: 0.001284
2. Training Time: 17 λεπτά

Training/Training in .eval() mode/Validation Curves:



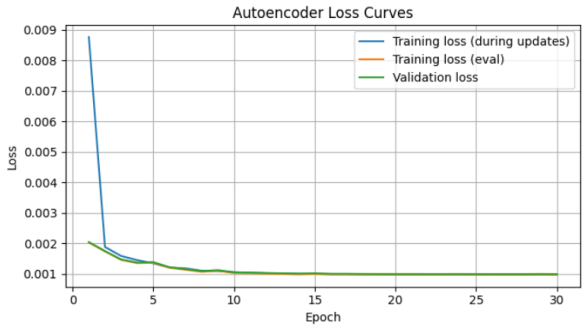
Setup 3:

1. Latent dimension μεγέθους 147
2. Width 64

Αποτελέσματα:

1. Minimum Loss: 0.000987
2. Training Time: 29 λεπτά

Training/Training in .eval() mode/Validation Curves:



Συμπεράσματα:

Οι Convolutional AEs, τουλάχιστον για τα δεδομένα setups, ξεπερνούν σε επίδοση τους MLP. Η σύγκριση αναφορικά με το latent dimension δεν είναι ακριβώς δίκαιη. Ωστόσο, ο ConvAE με latent dimension 96 ξεπερνά τον MLP με latent dimension 128. Επίσης σημειώνουμε πως η επίδραση που είχε η αύξηση του αρχικού width, είναι ελαφρώς μεγαλύτερη από αυτή του latent dimension. Βέβαια δεν αυξήθηκαν κατά τον ίδιο τρόπο. Αυτό που είναι βέβαιο είναι ότι αύξησε πολύ το training time.

6. Αξιολόγηση Μοντέλων στο Test Set.

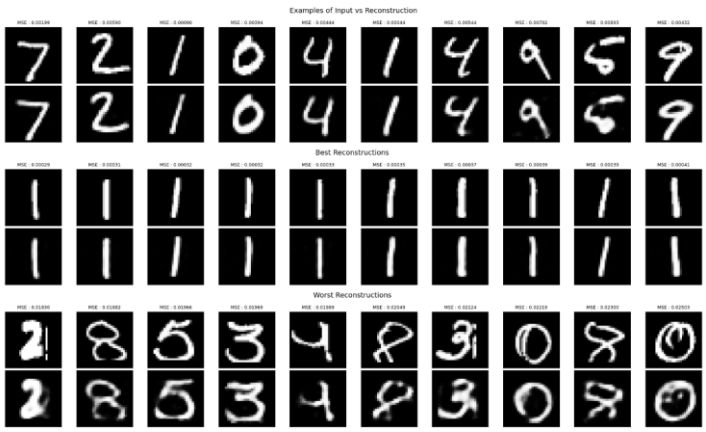
Επιλέχθηκε ο ισχυρότερος AE κάθε κατηγορίας. Για το PCA επιλέξαμε components όσα και το latent dimension του κάθε AE, ώστε να έχουμε δίκαιη (όσο γίνεται) σύγκριση.

6.1. MLP με Latent Dimension μεγέθους 128.

Loss: 0.004384

Fit Time: 7,25 sec.

Παραδείγματα Reconstructions (10 τυχαία , 10 καλύτερα , 10 χειρότερα):



Παρατηρούμε τα εξής. Αρχικά το μοντέλο αποδίδει περίπου όσο καλά όσο στην εκπαίδευση. Επίσης, οι καλύτερες αναπαραστάσεις αφορούν την κλάση 1. Λογικό, αφού έχει την απλούστερη δομή. Επίσης ενδιαφέρον είναι το πως οι χειρότερες αναπαραστάσεις αφορούν ιδιόμορφα instances με

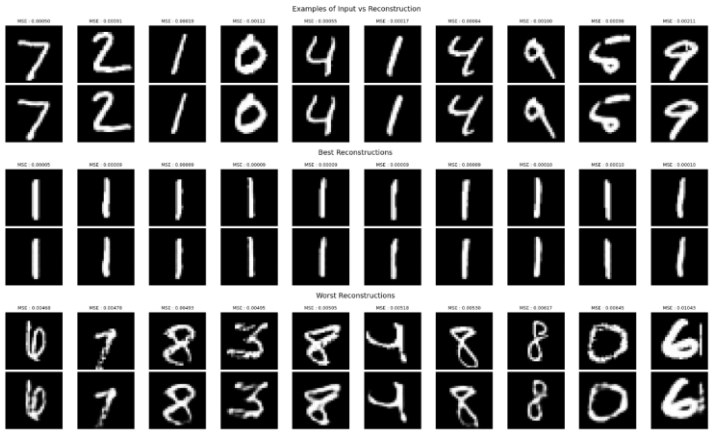
ιδιαιτερότητα στην δομή. Η ανακατασκευή είναι πολύ καλή κρίνοντας οπτικά. Είναι πιο blurred εκδοχές των αρχικών.

6.2. Convolutional AE με Latent Dimension μεγέθους 147 και Width 64.

Loss: 0.000961

Fit Time: 8,7 sec.

Παραδείγματα Reconstructions (10 τυχαία , 10 καλύτερα , 10 χειρότερα):



Οι παρατηρήσεις μας είναι ακριβώς ίδιες με της ενότητας 6.1. Μόνη διαφορά είναι πως τα reconstruction είναι πιο καλαίσθητα. Έχουν λιγότερο blur. Προσοχή: δεν ισχυριζόμαστε πως καλύτερο οπτικό αποτέλεσμα συνεπάγεται απαραίτητα καλύτερη ανακατασκευή. Ωστόσο αποτελεί πολύ καλό δείκτη.

6.3. PCA Reconstruction στο Test Set.

Ακολουθούν τα αποτελέσματα της PCA ανακατασκευής του test set.

```
PCA with 128 components.
Fit time:  0.57
Explained variance: 0.9365 MSE: 0.004208
PA with 147 components.
Fit time:  0.5
Explained varriance: 0.947 MSE: 0.00352
```

Για “apples to apples” σύγκριση, θα συγκρίνουμε με τους AEs αντίστοιχα με το latent dimension.

Το PCA Reconstruction αποδίδει καλύτερα από τον MLP. Με βάση όσα είχαμε πει προηγουμένως, η δεδομένη αρχιτεκτονική δεν έχει κάποιο σαφές πλεονέκτημα.

Ο Convolutional AE κυριαρχεί επί του PCA. Η διαφορά στην επίδοση δεν είναι trivial. Η δεδομένη αρχιτεκτονική πράγματι πλεονεκτεί έναντι της γραμμικής ανακατασκευής.

Όσο περισσότερα components, και κατά συνέπεια περισσότερο variance που εκφράζεται μέσω αυτών, τόσο μικρότερο το MSE της ανακατασκευής.

6.4. Επίδοση CNN στα Reconstructions.

Ενδιαφέρον ζητούμενο είναι το να δούμε το κατά πόσο τα reconstructions του κάθε μοντέλου αναγνωρίζονται από το NN. Ακολουθούν οι επιδόσεις του στο test set και σε reconstructions του MLP , του Conv AE και του PCA. Τα confusion matrices και τα αναλυτικά report δεν είχαν κάποιο ενδιαφέρον, οπότε περιλαμβάνονται μόνο στο Notebook.

Επίδοση στο Test Set: 99,31%

Επίδοση στο MLP AE (Lat. Dim. = 128): 98,76%

Επίδοση στο Conv AE (Lat. Dim = 147 , Width = 64): 99,27%

Επίδοση στο PCA (Components = 147): 99,16%

Πολύ καλά αποτελέσματα. Σχεδόν αμελητέα πτώση επίδοσης σε PCA και Conv. AE. Πολύ μικρή στο MLP AE.

7. Ανακατασκευή Επόμενου Ψηφίου.

Για τον σκοπό αυτό, δουλέψαμε ως εξής. Δημιουργούμε ένα τροποποιημένο dataset, το οποίο αντί για (image , label) , περιλαμβάνει στοιχεία της μορφής (image , target image , label). Αναθέτουμε, δηλαδή, σε κάθε εικόνα της κλάσης i , μία εικόνα της κλάσης i + 1 ως target. Το MSE loss υπολογίζεται πλέον με βάση ομοιότητα με εικόνες της επόμενης κλάσης. Έτσι, χωρίς καμία αλλαγή στην αρχιτεκτονική του μοντέλου, το ωθούμε να ανακατασκευάζει δείγματα επόμενου ψηφίου. Θεωρούμε πως μετά το 9 ακολουθεί το ψηφίο 0. Επίσης, η ανάθεση των targets γίνεται ντετερμινιστικά με βάση την θέση τους στο αρχικό dataset. Εάν τα ψηφία της κλάσης i + 1 δεν επαρκούν για να καλύψουν την i, κάνουμε circle από την αρχή. Δεν είναι, φυσικά, η μοναδική αρχιτεκτονική που θα μπορούσε να χρησιμοποιηθεί. Καθώς η διαδικασία είναι πιο απαιτητική, ξεκινάμε τα πειράματα με τον πιο ισχυρό από του AEs. Δεν ισχυριζόμαστε, ωστόσο, πως κατά κανόνα Autoencoder που ανακατασκευάζει καλύτερα ψηφία είναι και καλύτερος στον next digit generation.

7.1. Εκπαίδευση μοντέλου Balthazar για Next Digit Reconstruction.

Έγιναν τέσσερα διαφορετικά training runs. Καθώς το loss ήταν μεγάλο, επιλέχθηκε όλο μεγαλύτερο latent dimension και width.

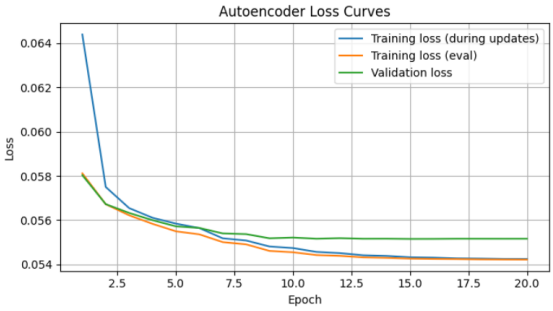
Setup 1:

1. Latent dimension μεγέθους 147
2. Width 64

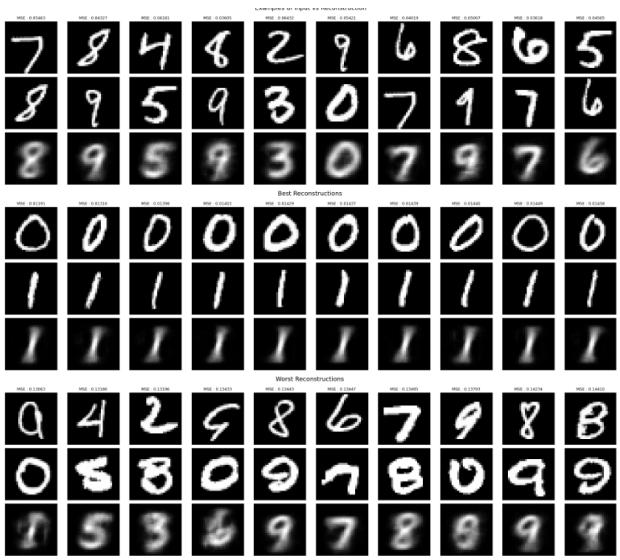
Αποτελέσματα:

1. Minimum loss: 0.055184
2. Training time 17 λεπτά

Training/Validation Curves:



Reconstruction Examples:



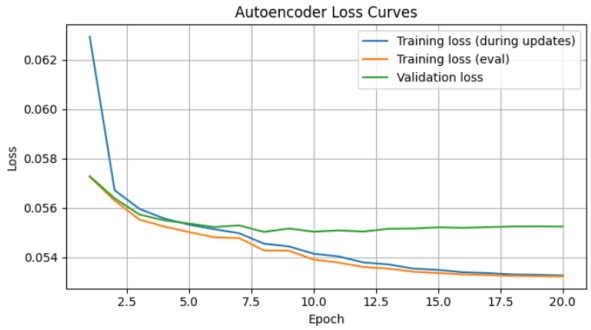
Setup 2:

1. Latent dimension μεγέθους 196
2. Width 96

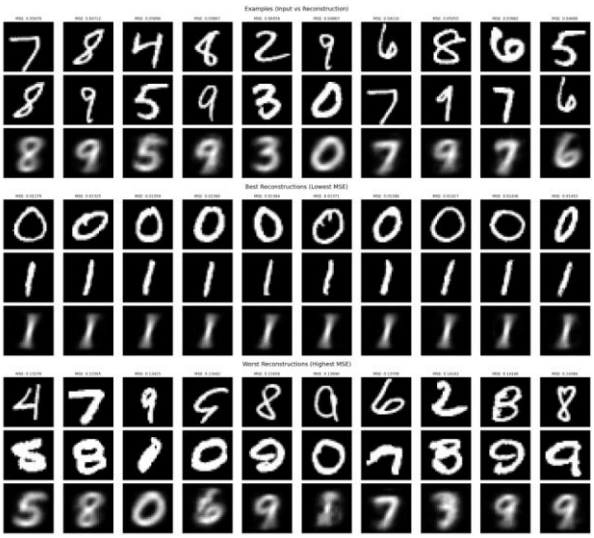
Αποτελέσματα:

1. Minimum loss: 0.055043
2. Training time 26 λεπτά

Training/Validation Curves:



Reconstruction Examples:



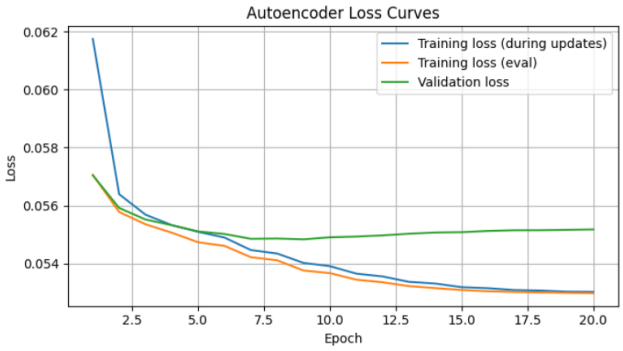
Setup 3:

1. Latent dimension μεγέθους 392
2. Width 96

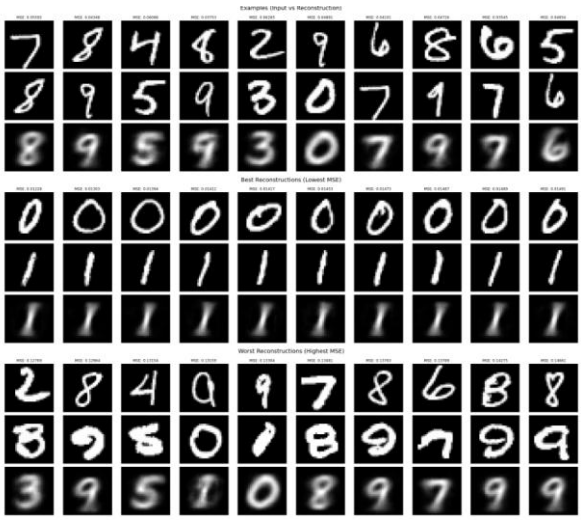
Αποτελέσματα:

1. Minimum loss: 0.055022
2. Training time 25 λεπτά

Training/Validation Curves:



Reconstruction Examples:



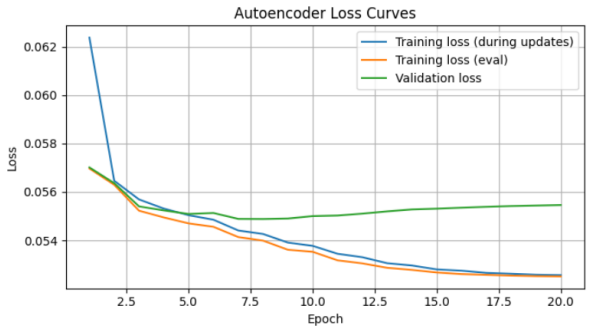
Setup 4:

1. Latent dimension μεγέθους 392
2. Width 128

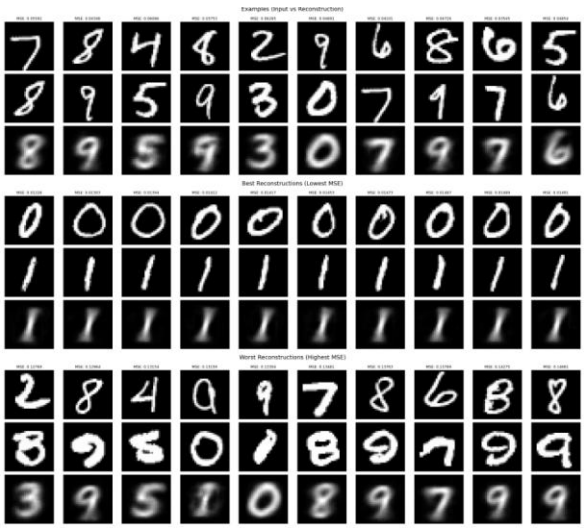
Αποτελέσματα:

1. Minimum loss: 0.054880
2. Training time 35 λεπτά

Training/Validation Curves:



Reconstruction Examples:



Συμπεράσματα:

Το MSE loss είναι αυξημένο κατά δεκάδες φορές σε σχέση με προηγουμένως. Οι ανακατασκευές είναι σαφώς πιο θολές. Επίσης μοιάζουν πολύ μεταξύ τους. Αυτό κατά πάσα πιθανότητα συμβαίνει εξαιτίας της δομής της εκπαίδευσης ανακατασκευής. Κάθε ψηφίο αντιστοιχίζεται σε target ανεξαρτήτως του περιεχομένου του. Έτσι παρόλο που πολλά targets είναι πιθανά για δεδομένο input, κάθε pairing γίνεται ντετερμινιστικά. Η χρήση MSE φαίνεται λοιπόν να ευνοεί το “averaging out” των κλάσεων. Εξ’ ου και τα θολά και ομοιόμορφα αποτελέσματα. Τα διαγράμματα έχουν κάποια ήπια σημάδια overfitting μετά από κάποια epochs, χωρίς ωστόσο αυτό να μας ανησυχεί. Κρατάμε τα καλύτερα βάρη με βάση το validation loss. Τέλος, τα μοντέλα δεν φαίνεται να έχουν σημαντικές διαφορές με βάση το loss ή τα οπτικά αποτελέσματα.

7.2. Επίδοση CNN στα Next Digit Reconstructions.

Αρχική επίδοση 99,28%.

Επίδοση στα Reconstructions του AE «βαρύτερου» AE: 98,07%

Επίδοση στα Reconstructions του «ελαφρύτερου AE: 96,84%

Παρόλο που το loss είναι πολύ μεγαλύτερο, το CNN πάλι τα καταφέρνει αξιοπρεπώς. Αυτό θα έλεγε κανείς πως είναι επιχείρημα υπέρ του robustness του δικτύου, παρά υπέρ της ποιότητας των ανακατασκευών. Τέλος, παρόλο που τα δύο μοντέλα δεν διαφέραν σημαντικά ως προς το loss, Βλέπουμε πως υπάρχει αξιοσημείωτη διαφορά στην επίδοση ταξινόμησης.